

Curriculum Skill Graph Pipeline: Steps 1 to 9

March 31, 2026

Contents

0.1	Step 1: Data Audit and Preprocessing	3
0.1.1	Outputs of this Step	4
0.1.2	Fields to Fill After Concluding This Step	6
0.1.3	Implementation Note and Current Run	7
0.2	Step 2: Global Pairwise Similarity and Cutoff Selection	9
0.2.1	Outputs of this Step	9
0.2.2	Fields to Fill After Concluding This Step	11
0.2.3	Implementation Note and Current Run	12
0.3	Step 3: Construction of the Review Table from the Selected Similarity Cutoffs . .	13
0.3.1	Outputs of this Step	14
0.3.2	Fields to Fill After Concluding This Step	15
0.3.3	Implementation Note and Current Run	16
0.4	Step 4: Harmonization of Skill Names and Construction of the New Harmonized Dataset	17
0.4.1	Outputs of this Step	18
0.4.2	Fields to Fill After Concluding This Step	20
0.4.3	Implementation Note and Current Run	21
0.5	Step 5: Definition of the Analytical Units and Construction of the Graph-Ready Node Tables	22
0.5.1	Outputs of this Step	23
0.5.2	Fields to Fill After Concluding This Step	25
0.5.3	Implementation Note and Current Run	26
0.6	Step 6: Construction of Full Weighted Similarity Graphs	27
0.6.1	Outputs of this Step	27
0.6.2	Fields to Fill After Concluding This Step	29
0.6.3	Implementation Note and Current Run	29
0.7	Step 7: Pooled-reference susceptibility sparsification and export of comparison-safe graph families	31
0.7.1	Outputs of this Step	33
0.7.2	Implementation Note and Current Run	34
0.8	Step 8: Representation comparison and selection of the primary graph family . .	36
0.8.1	Outputs of this Step	39
0.8.2	Implementation Note and Current Run	39
0.9	Step 9: Descriptive profiling and overlap analysis on the selected graph family . .	41
0.9.1	Outputs of this Step	44

0.9.2	Implementation Note and Current Run	45
-------	---	----

0.1 Step 1: Data Audit and Preprocessing

Before estimating semantic similarity between skills, the raw corpus must be converted into a clean, structurally consistent, and reproducible occurrence-level dataset. The purpose of this stage is not yet to harmonize skill labels or construct graphs, but to ensure that all downstream operations are applied to validated inputs. Because the skills were extracted automatically from curriculum documents with a large language model, the raw JSON may contain inconsistencies in formatting, missing values, duplicated records, malformed embedding vectors, and heterogeneous conventions across tracks, sections, programmes, and years. If such issues are not addressed at the beginning of the pipeline, later steps may confound genuine curricular structure with artifacts introduced during extraction or storage.

In this project, each record in the raw JSON is treated as a *skill occurrence*. A skill occurrence corresponds to one extracted skill associated with a specific curricular origin, identified through fields such as section, programme, year, and education type. At this stage, occurrences are not yet merged into canonical skills and no semantic decisions are made about equivalence across records. The audit therefore preserves the raw granularity of the extracted corpus while standardizing its representation and documenting any anomalies that may influence later analysis.

The first task in this step is schema validation. Each record must be checked for the presence of the expected fields: `skill_id`, `name`, `description`, `sections`, `programmes`, `year`, `edu_type`, `chunk_id`, `name_vector`, and `description_vector`. This validation has two objectives. First, it verifies whether the extraction pipeline produced a complete and internally consistent JSON structure. Second, it identifies whether some fields contain null values, empty strings, malformed objects, or unexpected types. In particular, the metadata fields `sections` and `programmes` must be inspected carefully to determine whether they are stored as single values or as lists. This distinction is important because the analytical units defined later in the project depend on a stable interpretation of curricular origin.

The second task is textual and metadata normalization. At this stage, the content of the skill names and descriptions should not be semantically rewritten, but their representation should be standardized. This includes trimming whitespace, standardizing character encoding, harmonizing capitalization when appropriate, and ensuring that metadata categories such as track names, programme labels, and section identifiers follow a single convention throughout the corpus. For example, if the same track or section appears under slightly different spellings, abbreviations, or formatting patterns, these variants should be mapped to a unique standardized form. All such transformations must be logged explicitly so that the preprocessing remains transparent and reversible.

The third task is vector integrity verification. Since the project relies on embeddings stored in `name_vector` and `description_vector`, these fields must be parsed and checked for numerical consistency. Every vector should be confirmed to be readable, numeric, and of the expected dimensionality. Records containing vectors of different lengths, non-numeric entries, truncated arrays, or empty values must be flagged. This step is essential because similarity estimation in the next stage assumes that all embeddings live in the same vector space and can therefore be compared directly. If malformed vectors are not identified here, the pairwise similarity tables generated in the harmonization step may become unreliable or fail computationally.

The fourth task is duplicate detection at the occurrence level. Exact duplicates may arise if the same skill was extracted more than once from the same curricular unit, or if duplicate records were introduced during data assembly. These duplicates should be identified using a strict rule based on the joint combination of the substantive and provenance fields, such as skill name, description, curricular origin, and embedding content. At this stage, the purpose is not to remove semantically similar skills across curricula, but only to detect records that appear to be literal or near-literal duplicates of the same occurrence. Any removals performed here must be conservative and fully documented in an exclusion or duplicate-resolution log.

The fifth task is coverage analysis. Once the dataset has been validated and standardized, a descriptive summary of the corpus must be produced. This summary should report the total number of skill occurrences, the number of valid and invalid records, the number of unique values for sections, programmes, years, and education types, and the distribution of records across these dimensions. This diagnostic is important for two reasons. First, it provides a factual inventory of the corpus before harmonization. Second, it reveals whether some curricular units are underrepresented, duplicated, or missing altogether, which could affect later comparison across sections, tracks, and pooled units.

The output of this stage is a cleaned master dataset at the skill-occurrence level. This cleaned dataset remains faithful to the original extraction, but it is structurally validated, consistently formatted, and ready for pairwise similarity analysis in the next step. No canonical skill names are introduced yet, and no graph is constructed at this stage. Instead, the main contribution of Step 1 is to create a reliable input base for all subsequent operations and to produce a transparent audit trail describing what was checked, what was standardized, what was excluded, and what still requires manual confirmation.

This step also establishes an important methodological principle for the rest of the pipeline: preprocessing must be non-destructive. Raw information should never be overwritten without trace. Whenever a value is standardized, repaired, or removed, the transformation must be documented in an accompanying log file. This guarantees that later harmonization and graph construction can be interpreted as analytical choices rather than hidden data modifications, thereby improving reproducibility and making the full workflow auditable.

0.1.1 Outputs of this Step

The main outputs of Step 1 consist of a cleaned occurrence-level dataset, a set of audit and validation reports, and a decision log for any issues requiring manual confirmation. These outputs are intended to document both the cleaned data and the full preprocessing logic applied before similarity estimation.

Files The following core files should be produced at the end of this step:

- `01_skills_occurrence_clean.json`: cleaned occurrence-level dataset, still with one row per extracted skill occurrence.
- `01_skills_occurrence_clean.csv`: tabular version of the cleaned occurrence-level dataset for manual inspection and downstream processing.
- `01_data_audit_summary.csv`: summary of completeness, validity, duplicates, and coverage.
- `01_vector_integrity_report.csv`: record-level report describing whether `name_vector` and `description_vector` are valid, numeric, and dimensionally consistent.
- `01_duplicate_report.csv`: exact or near-exact duplicate candidates detected at the occurrence level.
- `01_metadata_standardization_log.csv`: log of any standardization applied to fields such as `sections`, `programmes`, `edu_type`, or text formatting.
- `01_preprocessing_decision_log.csv`: decision log used only if the audit reveals ambiguities that require manual confirmation.

Tables to Produce The following tables should be generated as part of the audit and preprocessing documentation.

1. **Schema validation and missingness summary.** This table documents whether the expected fields are present and structurally valid. Its columns should be:

- field_name
- expected_type
- non_null_count
- null_count
- empty_string_count
- invalid_type_count

2. **Vector integrity summary.** This table summarizes the validity and dimensional consistency of the embedding fields. Its columns should be:

- vector_field
- expected_dimension
- valid_count
- invalid_count
- non_numeric_count
- wrong_dimension_count
- empty_count

3. **Duplicate occurrence report.** This table lists exact or near-exact duplicate skill occurrences detected during preprocessing. Its columns should be:

- duplicate_group_id
- skill_id
- name
- description
- sections
- programmes
- year
- edu_type
- duplicate_type
- recommended_action

4. **Coverage by curricular origin.** This table summarizes the distribution of skill occurrences across the curricular structure. Its columns should be:

- edu_type
- section
- programme
- year
- n_skill_occurrences
- n_unique_names

5. **Metadata standardization log.** This table records all standardization rules applied to metadata or text formatting. Its columns should be:

- `field_name`
- `raw_value`
- `standardized_value`
- `n_affected_records`
- `rule_applied`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the total number of raw records;
- the total number of valid records;
- the number of invalid or excluded records;
- whether `sections` and `programmes` are scalar or list-like in practice;
- the embedding dimension for `name_vector`;
- the embedding dimension for `description_vector`;
- whether all vectors are dimensionally and numerically consistent;
- whether exact duplicates exist;
- whether metadata labels require standardization;
- whether any curricular units are missing or underrepresented.

0.1.2 Fields to Fill After Concluding This Step

The following fields only need to be filled if the audit reveals ambiguity or requires a manual decision. In such cases, the decision should be recorded in `01_preprocessing_decision_log.csv` so that the preprocessing pipeline remains transparent and reproducible.

File `01_preprocessing_decision_log.csv`

Columns The decision log should contain the following columns:

- `decision_id`
- `issue_type`
- `affected_field`
- `detected_problem`
- `candidate_options`
- `chosen_option`
- `rationale`
- `n_affected_records`
- `status`

Typical Decisions That May Need to Be Filled The most common manual decisions expected at this stage are the following:

- **Storage rule for sections:**
 - preserve as list;
 - explode into one row per section;
 - convert to a single standardized label.
- **Storage rule for programmes:**
 - preserve as list;
 - explode into one row per programme;
 - convert to a single standardized label.
- **Policy for invalid vectors:**
 - exclude record;
 - repair from source if possible;
 - keep flagged but exclude from similarity calculations.
- **Policy for missing descriptions:**
 - keep record with missing description;
 - replace with empty string;
 - exclude record from combined-embedding analysis only.
- **Policy for exact duplicates:**
 - remove duplicate copy;
 - keep all and flag;
 - merge with provenance note.

0.1.3 Implementation Note and Current Run

The implemented Step 1 produced `01_skills_occurrence_clean.json`, `01_skills_occurrence_clean.csv`, and the diagnostics in `outputs/diagnostics`. In the current run, the raw input contained 895 records, the cleaned occurrence table contains 1,038 rows, both vector fields are 3072-dimensional, and no vectors were invalid.

The main documented deviations from the original written plan are the following:

- **Occurrence expansion rule.** The raw fields `sections` and `programmes` are stored as list-like strings, and 26 records could not be resolved cleanly by simple list broadcasting. The implemented occurrence table therefore uses the unique `section + programme` pairs encoded in `chunk_id`, together with a stable `section + programme -> year` lookup.
- **Duplicate detection scope.** The implemented preprocessing detects exact occurrence-level duplicates only. Near-duplicate detection was deferred intentionally to Step 2 semantic similarity analysis.
- **Standardization scope.** Normalization was kept conservative: trimming whitespace, collapsing repeated internal whitespace, and uppercasing metadata identifiers. Skill names and descriptions were not aggressively rewritten during preprocessing.

- **Occurrence-level schema.** The cleaned dataset uses singular fields `section`, `programme`, and `year`, while preserving raw source fields separately.
- **CSV serialization.** Array-valued fields such as vectors and grouped identifiers are stored as JSON strings in the CSV export.

0.2 Step 2: Global Pairwise Similarity and Cutoff Selection

After the data have been cleaned and validated, the next step is to compute semantic similarity across the global list of skills. At this stage, the analysis is not conducted separately by section, programme, track, or year. Instead, all skills are considered together in a single global pool. The purpose of this step is to generate a complete similarity table that will later be used to identify potential harmonization candidates.

Before pairwise similarity is computed, the occurrence-level output of Step 1 is converted into a deduplicated unique-skill base file. This file is named `01_unique_skills_vectors.csv` and contains one row per unique skill name, with three columns: `name`, `name_vector`, and `description_vector`. The purpose of this intermediate file is to remove repeated skills that differ only by curricular occurrence and to ensure that the global similarity analysis is performed over a unique list of skills rather than over repeated occurrence-level rows.

A simplifying rule is operationalized during the creation of this base file. Skills with exactly the same name do not need to be screened through pairwise similarity, because they are already assumed to refer to the same skill label. These exact-name matches are therefore separated from the similarity analysis and stored in a dedicated file for traceability. The pairwise similarity computation is then applied only to the rows of the unique-skill base file.

Two semantic representations are compared. The first uses only the embedding stored in `name_vector`. The second uses a combined representation based on `name_vector` and `description_vector`. The combined representation should be constructed in a fixed and transparent way. A simple and consistent choice is to normalize both vectors, concatenate them, and normalize the resulting combined vector before computing similarity. This ensures that both the short skill label and the skill description contribute to the final score.

The main operation of this step is the computation of cosine similarity for all unordered pairs of skills with different names. For each pair, two cosine similarity scores are recorded: one for the name-only representation and one for the combined name-plus-description representation. The result is a single global pairwise similarity table. This table is not yet a review file and does not yet imply that any two skills are equivalent. Its only purpose is to provide the quantitative basis for choosing a similarity cutoff.

Once the pairwise table has been generated, the next task is descriptive inspection. The distribution of similarity scores must be summarized for both representations. This includes the minimum, maximum, mean, median, standard deviation, and selected quantiles. The goal of this inspection is practical: it allows the researcher to see where potentially meaningful candidate pairs begin to emerge and to determine a workable cutoff for manual review. At the end of this step, the researcher chooses one cutoff for the name-only similarity and one cutoff for the combined name-plus-description similarity, or decides to retain only one of the two representations for the next stage.

This step therefore ends with a deliberate pause in the workflow. No skills are merged, no canonical names are assigned, and no human evaluation is performed yet. Instead, the main output is a global similarity table together with the cutoff choice that will define which candidate pairs move forward to the review stage.

Input The direct input to this step is therefore `01_unique_skills_vectors.csv`. If this file does not preserve an original identifier beyond the skill name, Step 2 may assign a stable temporary `skill_id` so that the pairwise similarity table remains uniquely indexable and reproducible.

0.2.1 Outputs of this Step

The outputs of Step 2 consist of the global similarity table, a record of exact-name matches, summary statistics for the similarity distributions, and a file in which the chosen cutoff values

can be recorded.

Files The following core files should be produced at the end of this step:

- `01_unique_skills_vectors.csv`: deduplicated base file created at the start of Step 2, with one row per unique skill name and columns `name`, `name_vector`, and `description_vector`; this is the direct input to the global similarity estimation.
- `02_exact_name_groups.csv`: list of skills that share exactly the same name and therefore do not need pairwise screening.
- `02_global_pairwise_similarity.csv`: one global table containing all unordered skill pairs with different names and their cosine similarity scores.
- `02_similarity_summary.csv`: summary statistics for the similarity distributions under both representations.
- `02_cutoff_selection.csv`: file in which the chosen cutoff values are recorded after inspection of the similarity table.

Tables to Produce The following tables should be generated as part of this step.

1. **Exact-name groups table.** This table records the skills that have exactly the same name during the creation of the unique-skill base file and are therefore excluded from pairwise screening. Its columns should be:

- `exact_name_group_id`
- `name`
- `n_skills`
- `skill_ids`

2. **Global pairwise similarity table.** This is the main output of the step. Each row corresponds to one unordered pair of skills with different names. Its columns should be:

- `pair_id`
- `skill_id_1`
- `skill_id_2`
- `name_1`
- `name_2`
- `cosine_name_only`
- `cosine_name_description`

If the unique-skill base file contains only `name`, `name_vector`, and `description_vector`, the `skill_id` fields may be stable temporary identifiers assigned within Step 2 for traceability.

3. **Similarity summary table.** This table summarizes the similarity distributions for both representations. Its columns should be:

- `representation`
- `n_pairs`
- `similarity_min`

- `similarity_p25`
- `similarity_median`
- `similarity_mean`
- `similarity_p75`
- `similarity_p90`
- `similarity_p95`
- `similarity_max`
- `similarity_std`

4. **Cutoff selection table.** This table stores the cutoff values chosen after inspection of the global similarity table. Its columns should be:

- `chosen_cutoff_name_only`
- `chosen_cutoff_name_description`
- `chosen_representation`
- `notes`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the file used as the direct input to the global similarity calculation;
- the total number of skills in the global list;
- the number of exact-name groups;
- the number of skills excluded from pairwise screening because of exact name matches;
- the total number of unordered pairs included in the similarity table;
- the distribution of cosine similarity scores for the name-only representation;
- the distribution of cosine similarity scores for the combined name-plus-description representation;
- whether temporary `skill_id` values were assigned for the pairwise table;
- the cutoff chosen for the name-only representation;
- the cutoff chosen for the combined representation;
- whether one of the two representations is preferred for the next step.

0.2.2 Fields to Fill After Concluding This Step

At the end of this step, the researcher must complete `02_cutoff_selection.csv`. This file records the cutoff values that will be used in Step 3 to build the review table.

File `02_cutoff_selection.csv`

Columns The file should contain the following columns:

- `chosen_cutoff_name_only`
- `chosen_cutoff_name_description`
- `chosen_representation`
- `notes`

Currently Recorded Values In the current run, `02_cutoff_selection.csv` contains the following values:

- `chosen_cutoff_name_only` = 0.70622
- `chosen_cutoff_name_description` = 0.701394
- `chosen_representation` = both
- `notes`: Pairs are retained for review if they pass at least one of the two cutoffs.

0.2.3 Implementation Note and Current Run

The implemented Step 2 uses `01_unique_skills_vectors.csv` exactly as the direct input file. In the current workflow, this file contains only the three columns `name`, `name_vector`, and `description_vector`, so Step 2 assigns stable temporary identifiers of the form `SK_0001` in order to keep the pairwise table reproducible and uniquely indexable.

The current run produced `02_exact_name_groups.csv`, `02_global_pairwise_similarity.csv`, `02_similarity_summary.csv`, and `02_cutoff_selection.csv`. The direct input contained 895 unique skill names, which yielded 400,065 unordered different-name pairs. Because the Step 2 input file was already deduplicated by exact name, `02_exact_name_groups.csv` is present only as a header-only file in the current run. This is not an error; it is simply the consequence of using a one-row-per-unique-name base file.

The currently recorded summary statistics and selections are:

- `similarity_p95` = 0.315562 for `name_only`;
- `similarity_p95` = 0.305355 for `name_description`;
- `chosen_cutoff_name_only` = 0.70622;
- `chosen_cutoff_name_description` = 0.701394;
- `chosen_representation` = both, with the practical rule that a pair is retained if it passes at least one cutoff.

0.3 Step 3: Construction of the Review Table from the Selected Similarity Cutoffs

After the global similarity table of unique skills has been generated in Step 2, the next stage is to construct the review table that will be used for manual inspection. The purpose of this step is operational and preparatory. It does not yet perform harmonization, assign canonical names, or modify the original skill records. Instead, it filters the global pairwise similarity table and retains only the unique-skill pairs whose similarity is sufficiently high to justify manual review. In the revised workflow, Step 2 was applied to a deduplicated global file of unique skills with embeddings. As a consequence, Step 3 also operates at the level of unique skills. Each row in the input similarity table corresponds to one unordered pair of distinct unique skills, together with two cosine similarity measures: one computed using `name_vector` only, and the other computed using the combined representation based on `name_vector` and `description_vector`. This design substantially reduces unnecessary manual work, because identical skill names were already collapsed before similarity estimation.

The filtering rule in this step follows directly from the cutoff values selected at the end of Step 2. A unique-skill pair is retained for manual review whenever its cosine similarity based on `name_vector` is greater than or equal to 0.70622, or its cosine similarity based on the combined name-plus-description representation is greater than or equal to 0.701394. These two thresholds are treated as alternative entry conditions into the review table. In other words, a pair is kept if it passes at least one of the two cutoffs:

$$\text{cosine_name_only} \geq 0.70622 \quad \text{or} \quad \text{cosine_name_description} \geq 0.701394.$$

The review table should remain simple and transparent. Each row corresponds to one unique-skill pair that passed the filtering rule. For each retained pair, the table should report the identifiers of the two unique skills, their names, the cosine similarity based on `name_vector`, the cosine similarity based on the combined representation, and two logical flags indicating whether the pair passed the name-only cutoff, the combined cutoff, or both. This makes it possible to see immediately which candidate pairs are supported by only one semantic representation and which are supported by both.

The function of this step is to convert the large global similarity table into a manageable candidate file for human judgment. It is therefore a filtering and organization stage rather than a substantive decision stage. No pair is labeled as equivalent or not equivalent at this point. Those decisions are introduced only afterward, during the manual review checkpoint. The output of Step 3 is simply the final table of unique-skill pairs that will be inspected manually.

This step also defines the transition to the harmonization stage. The review decisions collected after this step will be used in the following stage to create a new harmonized dataset containing all skills and their associated information. That new harmonized file must preserve the original extracted skill name and therefore should include a new field such as `old_name`, containing the original skill label before harmonization. It should also include a field such as `harmonized_name`, containing the final reviewed label, and a logical field such as `name_changed`, indicating whether the skill name was modified. In this way, the workflow remains transparent, reversible, and fully traceable.

At the end of this step, the workflow pauses again. The resulting review template is then examined manually, and the reviewer records whether each pair should be considered equivalent, not equivalent, or uncertain. These review decisions will only be used in the next step, when the new harmonized JSON file is constructed.

0.3.1 Outputs of this Step

The outputs of Step 3 consist of the filtered review table, a short summary of how many candidate pairs were retained, and an editable review template for manual completion. The harmonized JSON file is *not* produced in this step. It is produced only in the following step, after the manual review has been completed.

Files The following core files should be produced at the end of this step:

- `03_review_pairs_unique_skills.csv`: filtered table containing only the unique-skill pairs that passed at least one of the selected cutoffs.
- `03_review_summary.csv`: summary of the number of candidate pairs retained for manual review.
- `03_review_template_unique_skills.csv`: editable version of the filtered review table, with additional empty columns for manual evaluation.

Tables to Produce The following tables should be generated as part of this step.

1. **Review pairs table.** This is the main output of the step. Each row corresponds to one unique-skill pair retained for manual inspection. Its columns should be:

- `pair_id`
- `unique_skill_id_1`
- `unique_skill_id_2`
- `name_1`
- `name_2`
- `cosine_name_only`
- `cosine_name_description`
- `pass_name_only_cutoff`
- `pass_name_description_cutoff`

2. **Review summary table.** This table summarizes how many candidate pairs were retained under the selected filtering rule. Its columns should be:

- `summary_metric`
- `value`

Typical rows should include:

- number of pairs passing the name-only cutoff;
- number of pairs passing the combined cutoff;
- number of pairs passing both cutoffs;
- total number of review pairs.

3. **Manual review template.** This table is the editable version of the filtered review pairs file and will be completed after this step. Its columns should be:

- `pair_id`
- `unique_skill_id_1`

- `unique_skill_id_2`
- `name_1`
- `name_2`
- `cosine_name_only`
- `cosine_name_description`
- `review_decision`
- `review_notes`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the cutoff used for the name-only representation, namely 0.70622;
- the cutoff used for the combined name-plus-description representation, namely 0.701394;
- the number of pairs passing the name-only cutoff;
- the number of pairs passing the combined cutoff;
- the number of pairs passing both cutoffs;
- the total number of candidate pairs that will be manually reviewed.

0.3.2 Fields to Fill After Concluding This Step

The fields below are not filled during Step 3 itself. They are completed during the manual review checkpoint that follows this step.

File `03_review_template_unique_skills.csv`

Columns to Fill During Manual Review The following columns must be completed during the manual evaluation:

- `review_decision`
- `review_notes`

Suggested Review Labels A simple and sufficient set of labels for the manual review stage is:

- `equivalent`
- `not_equivalent`
- `uncertain`

Link to the Next Step Once the manual review has been completed, the next step will use the reviewed pairs to construct a new harmonized JSON file containing all skills and their associated information. This new file should preserve the original extracted name in a field such as `old_name`, store the reviewed harmonized label in a field such as `harmonized_name`, and include a logical field such as `name_changed`. This ensures that the harmonization process is explicit and that no original information is overwritten.

0.3.3 Implementation Note and Current Run

The current implementation produced `03_review_pairs_unique_skills.csv`, `03_review_summary.csv`, and `03_review_template_unique_skills.csv`. In the current run:

- 144 pairs pass the `name_only` cutoff;
- 57 pairs pass the `name_description` cutoff;
- 41 pairs pass both cutoffs;
- 160 pairs are retained for review in total.

For the present run, the cutoff itself was treated as the review decision: every retained pair was automatically labeled `equivalent` in `03_review_template_unique_skills.csv` with the note `automatic_equivalence_by_cutoff`. Step 3 therefore no longer pauses the workflow at a manual-review checkpoint in the current pipeline state.

0.4 Step 4: Harmonization of Skill Names and Construction of the New Harmonized Dataset

After the manual review of the filtered unique-skill pairs has been completed, the next step is to transform those review decisions into a harmonized skill naming layer and to propagate that layer back to the full skill dataset. This is the first stage at which the skill names are actually harmonized. The purpose of this step is therefore twofold. First, it converts the reviewed equivalence decisions into a harmonization mapping over the list of unique skills. Second, it uses that mapping to create a new JSON file containing all skills and their original information, together with additional fields that record the original name and the reviewed harmonized name. The input for this step is the manually completed review file produced after Step 3. In that file, each retained unique-skill pair has been labeled as **equivalent**, **not_equivalent**, or **uncertain**. Only the pairs labeled **equivalent** are used to construct harmonized groups. Pairs labeled **not_equivalent** are treated as evidence that the two unique skills must remain separate. Pairs labeled **uncertain** are not automatically merged and should remain separate unless an explicit additional decision is made outside the pairwise review file.

The first task in this step is to create equivalence groups among unique skills. Each unique skill is treated as a node, and each reviewed pair labeled **equivalent** is treated as an undirected connection between two nodes. The connected components of this reviewed equivalence graph define the preliminary harmonized groups. In practical terms, this means that if unique skill *A* was judged equivalent to *B*, and *B* equivalent to *C*, then all three unique skills are placed in the same harmonized group, unless an explicit manual exception is introduced. Each harmonized group is then assigned a new identifier, such as **harmonized_skill_id**.

The second task is to assign one harmonized name to each harmonized group. This is the substantive naming step of the workflow. For groups containing only one unique skill, the harmonized name is simply the original unique skill name. For groups containing more than one reviewed-equivalent unique skill, one harmonized label must be selected and assigned to the whole group. This harmonized label should be chosen in a clear and consistent manner. A practical and transparent rule is to select one representative name for the whole group and record that name explicitly in the harmonization mapping. Regardless of the naming rule adopted, it is essential that the original unique skill names remain preserved and are not overwritten without trace.

The third task is to construct the unique-skill harmonization mapping. This mapping is the bridge between the reviewed pairwise decisions and the full skill dataset. For every unique skill in the deduplicated global list, the mapping should record its original name, its harmonized group identifier, its harmonized name, and whether the name changed as a result of harmonization. This ensures that the harmonization logic remains explicit and can be audited independently of the full dataset.

The fourth task is to propagate the harmonization mapping back to the full skill file. At this point, the project returns from the unique-skill level to the complete dataset containing all skill records and their associated metadata. A new harmonized dataset must then be created. This new dataset should preserve all original fields already present in the skill records, including identifiers, descriptions, sections, programmes, year, education type, chunk information, and embeddings. In addition, the new dataset should include a field such as **old_name**, containing the original extracted skill name, a field such as **harmonized_name**, containing the final harmonized label assigned after review, and a logical field such as **name_changed**, indicating whether the name differs from the original. It is also strongly advisable to include **harmonized_skill_id** as a stable identifier for the harmonized skill group.

The resulting harmonized dataset is therefore not a replacement of the original data, but a new derived file. This distinction is methodologically important. The raw or cleaned dataset remains intact, while the harmonized dataset becomes the version used in all subsequent steps of the

project. In this way, the workflow remains reproducible, reversible, and fully documented. The harmonization can always be traced back to the reviewed pairwise decisions and to the original skill names.

The fifth task is to produce summary diagnostics for the harmonization stage. These diagnostics should report how many unique skills were merged into multi-name harmonized groups, how many unique skills remained unchanged, how many original skill records received a changed harmonized label, and how many harmonized groups were ultimately created. Such summary information is important because it quantifies the scale of the harmonization process before the graph construction stages begin.

At the end of this step, the project has a new harmonized JSON file that contains all skills and all original associated information, plus the new harmonization fields. This harmonized dataset becomes the main input for the subsequent stages of the workflow, including the definition of analytical units and the construction of similarity-based graphs.

0.4.1 Outputs of this Step

The outputs of Step 4 consist of the harmonization mapping at the unique-skill level, the new harmonized dataset at the full skill level, and a set of summary files documenting the effect of the harmonization procedure.

Files The following core files should be produced at the end of this step:

- `04_unique_skills_harmonization_mapping.csv`: mapping from each unique skill to its harmonized group and harmonized name.
- `04_unique_skills_harmonized.csv`: unique-skill-level table after harmonization.
- `04_skills_harmonized.json`: new full JSON file containing all skills and their original information, augmented with the harmonization fields.
- `04_skills_harmonized.csv`: tabular version of the full harmonized skill dataset.
- `04_harmonization_summary.csv`: summary of how many skills were merged, unchanged, or renamed.
- `04_harmonized_groups_summary.csv`: one-row-per-harmonized-group summary table.

Tables to Produce The following tables should be generated as part of this step.

1. **Unique-skill harmonization mapping table.** This is the main mapping table created from the reviewed decisions. Each row corresponds to one unique skill. Its columns should be:
 - `unique_skill_id`
 - `old_name`
 - `harmonized_skill_id`
 - `harmonized_name`
 - `name_changed`
 - `harmonization_status`
2. **Full harmonized skill table.** This table is the harmonized version of the full skill dataset and contains all original skill information plus the new harmonization fields. Its columns should include all original fields from the cleaned skill dataset, plus:

- old_name
- harmonized_name
- name_changed
- harmonized_skill_id

In the implemented workflow, the cleaned occurrence-level dataset already contains resolved occurrence identifiers and singular curricular fields. The full harmonized table should therefore retain the cleaned occurrence fields, notably:

- occurrence_id
- source_record_index
- occurrence_index_within_record
- skill_id
- name
- old_name
- harmonized_name
- name_changed
- harmonized_skill_id
- description
- section
- programme
- year
- edu_type
- chunk_ids
- chunk_id_count
- provenance_resolution_method
- name_vector_dimension
- description_vector_dimension
- name_vector
- description_vector
- raw_name
- raw_description
- raw_sections
- raw_programmes
- raw_year
- raw_edu_type
- raw_chunk_id

3. **Harmonization summary table.** This table summarizes the overall effect of the harmonization step. Its columns should be:

- summary_metric
- value

Typical rows should include:

- number of reviewed pairs labeled `equivalent`;
- number of unique skills;
- number of harmonized groups;
- number of multi-name harmonized groups;
- number of unique skills whose harmonized name differs from the original name;
- number of full skill records with `name_changed = TRUE`.

4. **Harmonized groups summary table.** This table provides one row per harmonized group. Its columns should be:

- `harmonized_skill_id`
- `harmonized_name`
- `n_unique_skills`
- `member_unique_skill_ids`
- `member_old_names`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the number of reviewed pairs labeled `equivalent`;
- the number of reviewed pairs labeled `not_equivalent`;
- the number of reviewed pairs labeled `uncertain`;
- the total number of unique skills before harmonization;
- the total number of harmonized groups after harmonization;
- the number of harmonized groups containing more than one unique skill;
- the number of unique skills whose harmonized name differs from the original name;
- the number of full skill records whose `harmonized_name` differs from `old_name`;
- the location and name of the new harmonized JSON file.

0.4.2 Fields to Fill After Concluding This Step

In most cases, Step 4 should be executable automatically once the manual review file has been completed. However, if a harmonized group contains several different original names, one representative label must be selected as the final harmonized name. If this choice cannot be determined automatically, it should be recorded explicitly before the final harmonized dataset is generated.

Optional File `04_harmonized_name_selection.csv`

Columns If needed, this file should contain the following columns:

- `harmonized_skill_id`
- `candidate_names`
- `chosen_harmonized_name`
- `selection_rationale`

Typical Cases Requiring Manual Completion The most common cases that may require explicit completion are:

- a harmonized group contains several different reviewed-equivalent names and one representative name must be chosen;
- an **uncertain** pair affects whether two provisional groups should remain separated or not;
- a naming convention must be standardized across several groups before writing the final harmonized JSON file.

Main New Fields Introduced in the Harmonized Dataset The new harmonized dataset created in this step must include at least the following additional fields:

- **old_name**: the original extracted skill name before harmonization;
- **harmonized_name**: the final reviewed skill label after harmonization;
- **name_changed**: logical field indicating whether **old_name** and **harmonized_name** differ;
- **harmonized_skill_id**: identifier of the harmonized skill group.

Role of This Step in the Workflow This step is the point at which the reviewed equivalence decisions are converted into a new harmonized JSON file containing all skills and all original associated information. From this point onward, the harmonized dataset becomes the main input for the rest of the project.

0.4.3 Implementation Note and Current Run

The implemented Step 4 produced `04_unique_skills_harmonization_mapping.csv`, `04_unique_skills_harmonized.csv`, `04_skills_harmonized.json`, `04_skills_harmonized.csv`, `04_harmonization_summary.csv`, and `04_harmonized_groups_summary.csv`. The full harmonized table preserves the cleaned occurrence-level fields and raw source fields, while adding **old_name**, **harmonized_name**, **name_changed**, and **harmonized_skill_id**.

The current run of Step 4 was executed before manual review decisions were entered in `03_review_template_unique_skills.csv`. As a result, the harmonization summary currently reports:

- 160 reviewed pairs labeled **equivalent**;
- 0 reviewed pairs labeled **not_equivalent**;
- 0 reviewed pairs labeled **uncertain**;
- 0 review pairs without a decision;
- 895 unique skills before harmonization;
- 746 harmonized groups after harmonization;
- 97 multi-skill harmonized groups;
- 149 unique skills with changed names;
- 188 full records with changed names.

The current Step 4 outputs therefore represent an active harmonization state rather than a no-merge baseline. The optional file `04_harmonized_name_selection.csv` is also populated in the current run because multi-skill groups required representative harmonized labels to be selected.

0.5 Step 5: Definition of the Analytical Units and Construction of the Graph-Ready Node Tables

After the harmonized dataset has been created in Step 4, the next stage is to define the analytical units on which the graphs will be constructed and to transform the harmonized full dataset into graph-ready node tables. The purpose of this step is not yet to create edges or sparsified graphs, but to establish exactly what a node is in each unit of analysis and to produce one clean node table per unit. This step is therefore the bridge between harmonization and graph construction. The harmonization in Step 4 was performed globally, using the complete set of unique skills. By contrast, the graphs in the later stages of the project are not built only once for the full dataset. They are built separately for different curricular units. In the present workflow, the analytical units are defined at three levels: sections, tracks, and the pooled dataset. A section-level unit contains all harmonized skill records belonging to one section across the relevant years. A track-level unit contains all harmonized skill records belonging to one education track. The pooled unit contains all harmonized skill records in the full dataset, regardless of section, programme, year, or track.

The first task in this step is therefore to define these analytical units explicitly and to assign each skill record in the harmonized dataset to the relevant unit or units. This assignment must be based on the harmonized full dataset created in Step 4 and must preserve the existing metadata, including at least section, programme, year, and education type. From this point onward, every subsequent graph construction step must use these same unit definitions so that the resulting graphs remain comparable.

The second task is to define the node rule. Within each analytical unit, a node corresponds to one `harmonized_skill_id`. This means that if the same harmonized skill appears multiple times inside the same unit, whether because it occurs in different years, different programmes, or different source records, it is represented by a single node in that unit-level graph. In other words, repeated occurrences of the same harmonized skill inside a unit are collapsed into one node. This is an essential rule because the graph is intended to represent the structure of skills within the curriculum, not the repetition count of identical labels.

The third task is to preserve provenance when repeated occurrences are collapsed. Even though a harmonized skill appears only once as a node within a unit, the aggregated node record must retain the information about where that node came from. This provenance should include the list of contributing `skill_id` values, the list of original names, the relevant sections, programmes, years, and education types represented in that node, and the number of source records that were collapsed. This ensures that the graph remains interpretable and that every node can be traced back to the underlying source material.

The fourth task is to define the node-level embedding that will later be used for graph construction. Since the similarity graphs will again be constructed using text embeddings, each node in each analytical unit must be assigned a single vector for the name-only representation and a single vector for the combined name-plus-description representation. When a node is formed from only one harmonized skill record in the unit, the node vector is simply the corresponding skill vector. When a node is formed from multiple contributing records in the same unit, the node vector should be computed as the average of the contributing vectors, followed by normalization. For the name-only representation, this means averaging the relevant `name_vector` values. For the combined representation, this means first constructing the combined vector using the same rule adopted in Step 2, then averaging those combined vectors across the contributing records, and finally normalizing the result. The same aggregation rule must be applied consistently to all units.

The output of this step is therefore a set of graph-ready node tables. Each row in these tables corresponds to one harmonized skill node in one analytical unit. These tables do not yet contain edges, similarity matrices, thresholds, or graph objects. Instead, they provide the clean node

universe from which the unit-level similarity graphs will be constructed in the next step.

A final methodological principle should be emphasized. Step 5 does not change the harmonized dataset itself. It creates derived node tables for graph construction. The harmonized full dataset created in Step 4 remains unchanged and continues to function as the reference dataset. The graph-ready node tables are therefore analytical products derived from that dataset, not replacements for it.

0.5.1 Outputs of this Step

The outputs of Step 5 consist of the formal definition of the analytical units, the graph-ready node tables for each unit family, and summary files describing how the harmonized records were collapsed into nodes.

Files The following core files should be produced at the end of this step:

- `05_analysis_units_definition.csv`: formal definition of the analytical units used for graph construction.
- `05_graph_ready_nodes_sections.csv`: graph-ready node table for all section-level units.
- `05_graph_ready_nodes_tracks.csv`: graph-ready node table for all track-level units.
- `05_graph_ready_nodes_pooled.csv`: graph-ready node table for the pooled unit.
- `05_node_aggregation_summary.csv`: summary of how many harmonized records were collapsed into nodes in each unit.
- `05_harmonized_node_provenance.csv`: provenance table linking each graph-ready node to its contributing source records.

Tables to Produce The following tables should be generated as part of this step.

1. **Analytical units definition table.** This table defines the units on which the graphs will later be constructed. Its columns should be:

- `unit_type`
- `unit_id`
- `unit_label`
- `definition_rule`
- `n_source_records`
- `n_graph_nodes`

Typical values of `unit_type` should be:

- `section`
- `track`
- `pooled`

2. **Graph-ready node table.** This is the main output of the step. Each row corresponds to one node in one analytical unit. Its columns should be:

- `unit_type`
- `unit_id`

- `harmonized_skill_id`
- `harmonized_name`
- `node_name_vector`
- `node_name_description_vector`
- `n_source_records`
- `n_source_years`
- `n_source_programmes`
- `n_source_sections`
- `n_source_edu_types`

3. **Node aggregation summary table.** This table summarizes how the harmonized full dataset was collapsed into graph-ready nodes for each unit. Its columns should be:

- `unit_type`
- `unit_id`
- `n_source_records`
- `n_harmonized_skill_ids`
- `n_graph_nodes`
- `n_collapsed_nodes`
- `mean_records_per_node`

4. **Node provenance table.** This table preserves the link between each graph-ready node and its contributing records in the harmonized dataset. Its columns should be:

- `unit_type`
- `unit_id`
- `harmonized_skill_id`
- `harmonized_name`
- `source_skill_ids`
- `source_old_names`
- `source_programmes`
- `source_sections`
- `source_years`
- `source_edu_types`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the full list of section-level units included in the analysis;
- the full list of track-level units included in the analysis;
- the definition of the pooled unit;
- the rule used to collapse repeated harmonized skills into one node per unit;
- the rule used to compute `node_name_vector`;

- the rule used to compute `node_name_description_vector`;
- the number of graph-ready nodes in each section-level unit;
- the number of graph-ready nodes in each track-level unit;
- the number of graph-ready nodes in the pooled unit;
- the number of nodes that aggregate more than one source record within each unit family.

0.5.2 Fields to Fill After Concluding This Step

In most cases, Step 5 should be executable automatically once the harmonized full dataset has been created in Step 4. However, if the unit definition or the node aggregation rule requires an explicit choice, that choice should be recorded before the graph-ready node tables are finalized.

Optional File `05_unit_and_aggregation_decisions.csv`

Columns If needed, this file should contain the following columns:

- `decision_id`
- `issue_type`
- `affected_component`
- `candidate_options`
- `chosen_option`
- `rationale`

Typical Cases Requiring Manual Completion The most common cases that may require explicit completion are:

- deciding whether section-level units are defined only by `section` or by a combination of `section` and another metadata field;
- deciding whether track-level units are defined only by `edu_type`;
- deciding whether node vectors are averaged across all contributing records or across distinct intermediate skill identifiers if such identifiers are preserved;
- deciding whether rare edge cases with incomplete provenance should be retained or excluded from the graph-ready node tables.

Role of This Step in the Workflow This step is the point at which the harmonized full dataset is converted into graph-ready node tables. From this point onward, the subsequent stages of the workflow no longer operate directly on raw or occurrence-level skill records, but on node tables that contain one harmonized skill node per analytical unit. These node tables become the direct input for similarity-based graph construction in the next step.

0.5.3 Implementation Note and Current Run

The implemented Step 5 produced `05_analysis_units_definition.csv`, `05_unit_and_aggregation_decisions.csv`, `05_graph_ready_nodes_sections.csv`, `05_graph_ready_nodes_tracks.csv`, `05_graph_ready_nodes_pooled.csv`, `05_node_aggregation_summary.csv`, and `05_harmonized_node_provenance.csv`. In the current run, the realized unit family consists of 13 section units, 2 track units, and 1 pooled unit, with 949 section-level node rows, 776 track-level node rows, 746 pooled node rows, and 2,471 provenance rows.

The optional decision file was created explicitly and records the substantive choices that were made:

- section units are defined by `section` across all years and programmes;
- track units are defined directly by `edu_type` values such as `CLS` and `GEN`;
- node vectors are constructed by averaging normalized occurrence vectors and renormalizing the result.

One practical deviation from the more expansive provenance wish-list is that not every provenance field is repeated in every Step 5 file. The provenance export stores grouped source identifiers, old names, programmes, sections, years, and education types, while occurrence counts are stored in the node tables and the raw `chunk_id` remains available in the Step 4 harmonized dataset rather than being duplicated in the compact provenance table.

Because the current Step 4 run contains no reviewed merges, the collapses performed in Step 5 primarily reflect repeated occurrences of the same harmonized label inside a unit rather than reviewed multi-name harmonization groups. In the current run, the pooled unit collapses 1,038 source records into 895 graph nodes.

0.6 Step 6: Construction of Full Weighted Similarity Graphs

After the graph-ready node tables have been created in Step 5, the next stage is to compute the full pairwise similarity structure within each analytical unit. No sparsification, thresholding, or topological pruning is applied yet. The purpose of this step is to construct the complete weighted graph family that will later serve as the basis for graph-level cutoff selection and comparison across units.

The direct inputs to this step are:

- 05_graph_ready_nodes_sections.csv
- 05_graph_ready_nodes_tracks.csv
- 05_graph_ready_nodes_pooled.csv

Each node table contains two vector representations:

- node_name_vector
- node_name_description_vector

These are the vectors used to compute graph edge weights.

For each analytical unit, Step 6 constructs two weighted similarity representations:

1. a graph based on `node_name_vector`;
2. a graph based on `node_name_description_vector`.

Within a given unit and representation, cosine similarity is computed for all node pairs. The graph is undirected, so each unordered pair of distinct nodes corresponds to exactly one edge in the full weighted graph. No threshold is imposed at this stage.

The current implementation stores this graph family in two complementary tabular forms. First, it writes long-format similarity matrices that retain all ordered row-column combinations, including diagonal self-similarities. Second, it writes full edge lists that retain all unordered node pairs with their cosine weights. Separate summary and metadata tables are then written so that graph size, file location, and similarity distributions can be compared across all units and representations without reopening the full matrices.

0.6.1 Outputs of this Step

Files The following core files should be produced at the end of this step:

- 06_similarity_matrices_sections.csv
- 06_similarity_matrices_tracks.csv
- 06_similarity_matrices_pooled.csv
- 06_weighted_edges_sections.csv
- 06_weighted_edges_tracks.csv
- 06_weighted_edges_pooled.csv
- 06_weighted_graph_summary.csv
- 06_weighted_graph_metadata.csv
- 06_graph_storage_decisions.csv

Tables to Produce The following tables should be generated as part of this step.

1. **Long-format similarity matrix table.** This table records the full within-unit cosine similarity matrix, including diagonal entries. Its columns should be:

- unit_type
- unit_id
- representation
- node_id_row
- node_id_col
- similarity_value

2. **Dense weighted edge list.** This table records every unordered pair of distinct nodes within a unit and representation. Its columns should be:

- unit_type
- unit_id
- representation
- harmonized_skill_id_1
- harmonized_skill_id_2
- harmonized_name_1
- harmonized_name_2
- edge_weight

3. **Weighted graph summary table.** This table summarizes the distribution of edge weights for each unit and representation. Its columns should be:

- unit_type
- unit_id
- representation
- n_nodes
- n_possible_edges
- edge_weight_min
- edge_weight_mean
- edge_weight_median
- edge_weight_std
- edge_weight_p75
- edge_weight_p90
- edge_weight_p95
- edge_weight_max

4. **Weighted graph metadata table.** This table records the identifying and storage metadata for every graph in the family. Its columns should be:

- graph_id
- unit_type
- unit_id

- `representation`
- `n_nodes`
- `n_possible_edges`
- `matrix_file`
- `edge_file`
- `notes`

Information That Should Be Recorded At the end of this step, the following information should be known and explicitly documented:

- the number of graphs created for section, track, and pooled units;
- the number of nodes in each graph;
- the number of possible undirected edges in each graph;
- the distribution of edge weights in each graph under both representations;
- the storage format used for the similarity matrices;
- the storage format used for the edge lists;
- whether diagonal self-similarities are retained in the matrix exports;
- whether all unordered node pairs are retained in the dense edge lists;
- the file locations of the matrix and edge exports for each graph.

0.6.2 Fields to Fill After Concluding This Step

Step 6 should generally be executable automatically once the Step 5 node tables exist. If the project requires an explicit written record of graph-storage choices, those decisions should be captured in `06_graph_storage_decisions.csv`.

0.6.3 Implementation Note and Current Run

The current implementation produced all nine files listed above. The dense graph family is represented by the combination of long-format similarity matrices, dense unordered edge lists, graph summaries, graph metadata, and `06_graph_storage_decisions.csv`. No separate serialized graph-object file is written in the current workflow; this is the main practical deviation from a more abstract formulation that would refer to a separate graph-object export.

The current run produced 32 graph summaries in total:

- 26 section graphs;
- 4 track graphs;
- 2 pooled graphs.

The recorded storage decisions are:

- long-format similarity matrices are used instead of wide matrices;
- dense edge lists retain all unordered node pairs, including negative cosine similarities;

- graph metadata are stored in a separate file rather than embedded in each edge export.

An important current-state note is that Step 6 has been run on node tables built from the active Step 4 harmonization layer in which all cutoff-retained Step 3 pairs were merged automatically. The resulting full weighted graphs therefore reflect the current post-harmonization state of the workflow rather than the earlier no-merge baseline.

0.7 Step 7: Pooled-reference susceptibility sparsification and export of comparison-safe graph families

The weighted candidate graphs inherited from the previous steps remain too dense to be interpreted directly as curriculum networks. The purpose of the present step is therefore to convert each representation family into a sparse graph basis using the susceptibility criterion, while preserving cross-unit comparability. In the restarted workflow, sparsification is performed primarily in the pooled reference universe and only secondarily at the local unit level for diagnostic purposes. This distinction is essential: the pooled-reference sparse graphs are used for the main comparisons across years, programmes, sections, and tracks, whereas local unit-specific sparse graphs are retained only as robustness outputs.

Let

$$\mathcal{R}_{\text{spec}} = \{\text{name_only}, \text{fusion}, \text{concat}\}$$

denote the set of candidate representation families in the methodological specification. For each family $r \in \mathcal{R}_{\text{spec}}$, Step 6 fixes a pooled reference form, so that detailed and non-detailed files are no longer treated as separate substantive representations at this stage.

In the current implementation, however, the set of families available on disk is a subset of the full specification. We therefore distinguish the available family set

$$\mathcal{R}_{\text{avail}} \subseteq \mathcal{R}_{\text{spec}}.$$

In the present run,

$$\mathcal{R}_{\text{avail}} = \{\text{name_only}, \text{concat}\}.$$

The concatenated name-plus-description representation is produced upstream under the source label `name_description` and is exported in Step 7 under the family label `concat`. The family `fusion` remains part of the methodological specification but is not available in the current run. For each available family $r \in \mathcal{R}_{\text{avail}}$, let

$$G_{\text{ref}}^{(r)} = (V^{(\text{all_tracks})}, E^{(r)}, w^{(r)})$$

denote the pooled weighted candidate graph built on the common all-tracks node universe and on the reference form selected earlier.

For each family r , the set of candidate thresholds is defined as the set of unique observed edge weights,

$$\Theta^{(r)} = \{w_{ij}^{(r)} : \{i, j\} \in E^{(r)}\}.$$

For a given threshold $\tau \in \Theta^{(r)}$, the thresholded pooled graph is

$$G_{\text{ref}, \tau}^{(r)} = (V^{(\text{all_tracks})}, E_{\tau}^{(r)}, w_{\tau}^{(r)}),$$

with

$$E_{\tau}^{(r)} = \{\{i, j\} \in E^{(r)} : w_{ij}^{(r)} \geq \tau\},$$

and where surviving edges retain their original weights. The associated binary version is defined by

$$A_{\tau, ij}^{(r)} = \mathbb{1}(w_{ij}^{(r)} \geq \tau).$$

As in the previous methodological record, threshold selection is based on the susceptibility of finite connected components after excluding the largest one. Let

$$C_1^{(r)}(\tau), C_2^{(r)}(\tau), \dots, C_{K(\tau)}^{(r)}(\tau)$$

denote the connected components of $G_{\text{ref},\tau}^{(r)}$ ordered by size, with

$$|C_1^{(r)}(\tau)| \geq |C_2^{(r)}(\tau)| \geq \dots$$

For each component size s , define

$$n_s^{(r)}(\tau) = \#\{k \geq 2 : |C_k^{(r)}(\tau)| = s\},$$

that is, the number of finite components of size s after excluding the largest connected component. The susceptibility curve is then

$$\chi^{(r)}(\tau) = \frac{\sum_s s^2 n_s^{(r)}(\tau)}{\sum_s s n_s^{(r)}(\tau)},$$

with the convention that $\chi^{(r)}(\tau) = 0$ whenever no finite components remain outside the largest component.

The set of susceptibility-maximizing thresholds for family r is

$$\mathcal{M}_r = \arg \max_{\tau \in \Theta^{(r)}} \chi^{(r)}(\tau).$$

Because tied maxima may occur, the restarted workflow does not automatically choose the sparsest maximizer. Instead, tie resolution is performed through a pooled-reference usability rule. Let

$$\lambda_r(\tau) = \frac{|C_1^{(r)}(\tau)|}{|V(\text{all_tracks})|}$$

denote the largest-connected-component share, and let

$$\iota_r(\tau) = \frac{|I^{(r)}(\tau)|}{|V(\text{all_tracks})|}$$

denote the isolate share at threshold τ , where $I^{(r)}(\tau)$ is the set of isolated nodes in $G_{\text{ref},\tau}^{(r)}$. Define the set of admissible maximizers

$$\mathcal{U}_r = \{\tau \in \mathcal{M}_r : \lambda_r(\tau) \geq \lambda_{\min} \text{ and } \iota_r(\tau) \leq \iota_{\max}\},$$

where $\lambda_{\min}$ and $\iota_{\max}$ are fixed *ex ante*. The operational pooled cutoff for family r is then defined as

$$\tau_r^* = \begin{cases} \max \mathcal{U}_r, & \text{if } \mathcal{U}_r \neq \emptyset, \\ \min \mathcal{M}_r, & \text{if } \mathcal{U}_r = \emptyset. \end{cases}$$

Thus, whenever susceptibility ties occur, the retained cutoff is the largest maximizing threshold that still satisfies the pooled usability constraints; if none of the maximizing thresholds satisfies these constraints, the least aggressive maximizing threshold is selected. This fallback preserves the topological-transition logic while avoiding unnecessary over-sparsification within the maximizing set.

The resulting pooled sparse reference graph for family r is

$$H_{\text{all}}^{(r)} = G_{\text{ref},\tau_r^*}^{(r)}.$$

This graph is the primary sparse object used in the subsequent representation-comparison step. All lower-level analytical units are then obtained as induced subgraphs of this same pooled sparse

reference graph. Let V_t , V_p , and $V_{p,y}$ denote, respectively, the node sets associated with track t , programme p , and programme-year unit (p, y) . The corresponding sparse graphs are defined as

$$H_t^{(r)} = H_{\text{all}}^{(r)}[V_t], \quad H_p^{(r)} = H_{\text{all}}^{(r)}[V_p], \quad H_{p,y}^{(r)} = H_{\text{all}}^{(r)}[V_{p,y}],$$

where $H[\cdot]$ denotes node-induced subgraph extraction. Because all of these graphs are inherited from the same thresholded pooled universe, they remain directly comparable across tracks, programmes, sections, and years.

For diagnostic purposes only, the same susceptibility procedure may also be run separately within each analytical unit A . Let

$$\tau_{A,\text{diag}}^{(r)}$$

denote the local susceptibility cutoff obtained for unit A under family r , and let

$$H_{A,\text{diag}}^{(r)}$$

be the corresponding locally thresholded graph. These local sparse graphs are retained as robustness outputs and may be used to assess how sensitive the results are to local thresholding. However, they are not used as the primary basis for cross-unit substantive comparison, since unit-specific thresholds would confound curricular differences with sparsification differences.

For each available family $r \in \mathcal{R}_{\text{avail}}$, the following pooled Step 7 diagnostics are retained:

$$\tau_r^*, \quad \chi^{(r)}(\tau_r^*), \quad |\mathcal{M}_r|, \quad |E(H_{\text{all}}^{(r)})|, \quad \lambda_r(\tau_r^*), \quad \iota_r(\tau_r^*),$$

together with the connected-node share

$$\nu_r = \frac{|V^{(\text{all_tracks})}| - |I^{(r)}(\tau_r^*)|}{|V^{(\text{all_tracks})}|},$$

the retained-edge proportion, and the full susceptibility curve. These quantities provide the direct input for the representation decision in Step 8.

The output of Step 7 therefore consists of two layers. The first is the primary pooled-reference sparse graph family

$$\left\{ H_{\text{all}}^{(r)}, H_t^{(r)}, H_p^{(r)}, H_{p,y}^{(r)} \right\}_{r \in \mathcal{R}_{\text{avail}}},$$

which forms the comparison-safe basis for the representation decision and all later substantive analyses. The second is a diagnostic family of locally thresholded unit-specific graphs, retained only for robustness assessment. This separation ensures that the main pipeline preserves comparability while still documenting the behavior of the susceptibility rule at the local level.

0.7.1 Outputs of this Step

The implemented Step 7 writes both pooled-reference and local-diagnostic outputs under `outputs/graphs_sparse`. The current file set is:

- `07_pooled_reference_cutoffs.csv`
- `07_pooled_reference_susceptibility_curves.csv`
- `07_pooled_reference_sparse_graphs_all_tracks.csv`
- `07_pooled_reference_sparse_graphs_sections.csv`
- `07_pooled_reference_sparse_graphs_tracks.csv`
- `07_pooled_reference_sparse_graphs_programmes.csv`

- 07_pooled_reference_sparse_graphs_programme_years.csv
- 07_pooled_reference_sparse_graph_summary.csv
- 07_local_diagnostic_cutoffs.csv
- 07_local_diagnostic_susceptibility_curves.csv
- 07_local_diagnostic_sparse_graphs_sections.csv
- 07_local_diagnostic_sparse_graphs_tracks.csv
- 07_local_diagnostic_sparse_graphs_programmes.csv
- 07_local_diagnostic_sparse_graphs_programme_years.csv
- 07_local_diagnostic_summary.csv
- 07_step7_decision_log.csv

The pooled-reference cutoff table contains one selected cutoff per available representation family. The pooled-reference summary table contains one row for each exported comparison-safe sparse graph: the all-tracks graph plus the induced section, track, programme, and programme-year subgraphs. The local-diagnostic cutoff and summary tables contain one row for each locally thresholded lower-level unit graph. The file `07_step7_decision_log.csv` is an implementation log recording the threshold-selection path and fallback behavior within Step 7; it is not the formal representation decision matrix, which is introduced only in Step 8.

0.7.2 Implementation Note and Current Run

The current run processed the two available representation families `name_only` and `concat`. The upstream label `name_description` is mapped to the Step 7 family label `concat`; `fusion` is not available in the present pipeline state.

The fixed pooled-reference usability parameters used in the current implementation are:

- $\lambda_{\min} = 0.90$
- $\iota_{\max} = 0.10$

These values are deliberately strict and act as an admissibility screen within the set of susceptibility maximizers. In the present run, they do not admit any maximizing threshold for either available family, so the fallback branch of the rule is activated in both cases.

The implemented program wrote:

- 2 pooled-reference cutoffs;
- 341,141 pooled-reference susceptibility-curve rows;
- 474,034 local-diagnostic susceptibility-curve rows;
- 126 pooled-reference sparse-graph summary rows;
- 124 local-diagnostic summary rows.

The selected pooled-reference cutoffs in the current run are:

- `name_only`: $\tau^* = 0.585136$

- **concat:** $\tau^* = 0.519089$

In both cases the admissible-maximizer set $\mathcal{U}_r$ is empty under the current $(\lambda_{\min}, \iota_{\max})$ rule, so the selected pooled cutoff is the least aggressive susceptibility maximizer rather than the largest admissible maximizer. The resulting pooled sparse graphs are therefore still fragmented in the current run. For the selected pooled thresholds, the retained pooled graphs have:

- **name_only:** 379 retained pooled edges, largest-component share 0.0791, isolate share 0.4812
- **concat:** 739 retained pooled edges, largest-component share 0.2681, isolate share 0.2534

The local-diagnostic cutoffs range from 0.294399 to 0.613096 across sections, tracks, programmes, and programme-year units. These local graphs are retained only for robustness comparison; the comparison-safe graph family used in the next step remains the pooled-reference family induced from the two pooled cutoffs above.

0.8 Step 8: Representation comparison and selection of the primary graph family

Once the pooled-reference sparse graph families have been exported in Step 7, the next task is to select the representation family that will serve as the primary basis for all subsequent analyses. This is a model-selection stage. Its purpose is not yet to interpret curricular differences, but to decide which available representation family yields the most useful and most comparable curriculum graph basis under the pooled-reference sparsification rule.

The representation decision is made at the pooled all-tracks level, not separately within each section, programme, track, or programme-year unit. This is a deliberate design choice. If different analytical units were allowed to select different representation families, then later differences between curricula would be confounded with differences in representation choice. The formal representation decision is therefore made once, in a common pooled reference space, and the lower-level units are used only to audit the consistency of that choice.

Let

$$\mathcal{R}_{\text{spec}} = \{\text{name_only}, \text{fusion}, \text{concat}\}$$

denote the full methodological family set, and let

$$\mathcal{R}_{\text{avail}} \subseteq \mathcal{R}_{\text{spec}}$$

denote the subset of families actually available in the current run. For each

$$r \in \mathcal{R}_{\text{avail}},$$

Step 7 provides a pooled sparse reference graph

$$H_{\text{all}}^{(r)} = G_{\text{ref}, \tau_r^*}^{(r)},$$

together with induced pooled-reference subgraphs at the track, programme, section, and programme-year levels.

(i) Eligibility screen. Before computing the decision scores, each available family is screened for pooled-reference usability. Let

$$\lambda_r = \frac{|C_1^{(r)}|}{|V(\text{all_tracks})|}$$

denote the largest-connected-component share of the pooled sparse graph $H_{\text{all}}^{(r)}$, and let

$$\iota_r = \frac{|I^{(r)}|}{|V(\text{all_tracks})|}$$

denote its isolate share. A family is said to pass the usability screen if

$$\text{Eligible}(r) = \mathbb{1}(\lambda_r \geq \lambda_{\min} \text{ and } \iota_r \leq \iota_{\max}) = 1.$$

Let

$$\mathcal{R}_{\text{elig}} = \{r \in \mathcal{R}_{\text{avail}} : \text{Eligible}(r) = 1\}$$

be the eligible family set. The formal decision set is then

$$\mathcal{R}_{\text{dec}} = \begin{cases} \mathcal{R}_{\text{elig}}, & \text{if } \mathcal{R}_{\text{elig}} \neq \emptyset, \\ \mathcal{R}_{\text{avail}}, & \text{if } \mathcal{R}_{\text{elig}} = \emptyset. \end{cases}$$

Thus, if at least one family passes the usability screen, only eligible families are allowed to compete for selection. If none passes, the comparison is still carried out over all available families, but the final choice is explicitly flagged as a fallback decision under globally fragmented pooled graphs.

(ii) Formal pooled decision matrix. The primary representation decision is based on two criteria computed from the pooled Step 7 outputs: *structural adequacy* and *parsimony*. The objective is to select one primary family

$$r^* \in \mathcal{R}_{\text{dec}},$$

while retaining the remaining families as sensitivity families.

Structural adequacy. A useful curriculum graph should display a clear topological transition, preserve a substantial share of nodes in the retained core, and avoid excessive isolation. For the pooled graph $H_{\text{all}}^{(r)}$, define

$$\chi_r^* = \chi^{(r)}(\tau_r^*), \quad \lambda_r = \frac{|C_1^{(r)}|}{|V(\text{all_tracks})|}, \quad \nu_r = \frac{|V(\text{all_tracks})| - |I^{(r)}|}{|V(\text{all_tracks})|}.$$

Here χ_r^* is the peak susceptibility at the selected pooled cutoff, λ_r is the largest-component share, and ν_r is the connected-node share.

After min-max normalization across the decision set $\mathcal{R}_{\text{dec}}$, define

$$q_{\text{struct}}^{(r)} = \omega_1 \tilde{\chi}_r^* + \omega_2 \tilde{\lambda}_r + \omega_3 \tilde{\nu}_r,$$

with fixed ex ante weights

$$\omega_1 = 0.4, \quad \omega_2 = 0.3, \quad \omega_3 = 0.3.$$

If a component is constant over all $r \in \mathcal{R}_{\text{dec}}$, its normalized value is set to 1 for all families. This score rewards families that produce a strong topological transition and a more usable pooled post-threshold graph.

Parsimony. The selected family should also be reasonably sparse after thresholding. Let

$$m_r = |E(H_{\text{all}}^{(r)})|$$

denote the number of retained pooled edges. Define the normalized inverse retained-edge score

$$\tilde{\pi}_r = \frac{\max_{q \in \mathcal{R}_{\text{dec}}} m_q - m_r}{\max_{q \in \mathcal{R}_{\text{dec}}} m_q - \min_{q \in \mathcal{R}_{\text{dec}}} m_q},$$

with the convention that $\tilde{\pi}_r = 1$ if all retained-edge counts are equal. Higher values correspond to greater parsimony.

(iii) Composite pooled score and scenario totals. The main pooled representation score is defined as

$$\text{Score}(r) = \alpha q_{\text{struct}}^{(r)} + \beta \tilde{\pi}_r,$$

where

$$\alpha, \beta \geq 0, \quad \alpha + \beta = 1.$$

In the restarted workflow, the ex ante weights are fixed as

$$\alpha = 0.75, \quad \beta = 0.25.$$

This gives priority to structural usefulness while still rewarding sparsity. The primary representation is then selected as

$$r^* = \arg \max_{r \in \mathcal{R}_{\text{dec}}} \text{Score}(r).$$

For transparency, three scenario totals are also reported using the same two pooled criteria but different weightings:

$$\begin{aligned}\text{Richness}(r) &= 0.85 q_{\text{struct}}^{(r)} + 0.15 \tilde{\pi}_r, \\ \text{Simplicity}(r) &= 0.60 q_{\text{struct}}^{(r)} + 0.40 \tilde{\pi}_r, \\ \text{Balanced}(r) &= 0.75 q_{\text{struct}}^{(r)} + 0.25 \tilde{\pi}_r.\end{aligned}$$

These scenario totals do not replace the main pooled score. Rather, they document whether the selected family remains preferred under alternative emphases on structural richness versus sparsity. For each family, the following additional summary quantities are retained:

$$\overline{\text{Scenario}}(r) = \frac{\text{Richness}(r) + \text{Simplicity}(r) + \text{Balanced}(r)}{3},$$

together with the number of scenario wins across the three totals.

(iv) Unit-level consistency audit. Although the formal representation choice is made at the pooled level, it is useful to verify that the selected family also behaves well across lower-level units. This is done through a unit-level consistency audit based on the pooled-reference induced subgraphs from Step 7. Let

$$\mathcal{U} = \{\text{sections}, \text{tracks}, \text{programmes}, \text{programme_years}\}$$

denote the set of lower-level unit classes, and let $\mathcal{A}_u$ be the set of analytical units in class u , each represented by its pooled-reference induced graph $H_A^{(r)}$.

For each unit class u , the audit uses the beneficial metric set

$$\mathcal{M}_+ = \{\text{LCC share}, \text{connected-node share}, \text{density}\}$$

and the cost metric set

$$\mathcal{M}_- = \{\text{isolate share}, \text{component count}\}.$$

For a given metric m and unit class u , define the dominance share of family r as

$$D_{m,u}^{(r)} = \frac{1}{|\mathcal{A}_u|} \sum_{A \in \mathcal{A}_u} \mathbb{1} \left(m(H_A^{(r)}) = \underset{q \in \mathcal{R}_{\text{dec}}}{\text{opt}} m(H_A^{(q)}) \right),$$

where opt denotes $\max$ for $m \in \mathcal{M}_+$ and $\min$ for $m \in \mathcal{M}_-$. Ties at the unit level are allowed and are credited to all tied best families. The overall consistency audit score is then

$$q_{\text{cons}}^{(r)} = \frac{1}{|\mathcal{U}|(|\mathcal{M}_+| + |\mathcal{M}_-|)} \sum_{u \in \mathcal{U}} \left(\sum_{m \in \mathcal{M}_+} D_{m,u}^{(r)} + \sum_{m \in \mathcal{M}_-} D_{m,u}^{(r)} \right).$$

This audit score is reported as supporting evidence only. It is not part of the formal pooled selection score, because the primary methodological objective is to choose one common representation family for all downstream comparisons, not to allow different unit types to select different representations.

Outputs of the representation-selection step. The outputs of Step 8 are fourfold. First, a formal pooled decision matrix reports, for each family in $\mathcal{R}_{\text{dec}}$, the selected cutoff, peak susceptibility, largest-component share, connected-node share, isolate share, retained pooled edges, structural score, parsimony score, composite pooled score, eligibility flag, and final rank. Second, a scenario table reports the richness, simplicity, and balanced totals together with average scenario score and scenario-win count. Third, a unit-level consistency table reports the lower-level audit results by unit class. Fourth, a one-row selection record freezes the primary family r^* . The output of this step is therefore a frozen primary graph family

$$\mathcal{H}^{(r^*)} = \left\{ H_{\text{all}}^{(r^*)}, H_t^{(r^*)}, H_p^{(r^*)}, H_{p,y}^{(r^*)} \right\},$$

which will be used for the main descriptive, topological, and comparison-oriented analyses in the subsequent stages. The remaining available families are retained as sensitivity families. Importantly, this decision pertains to the representation layer only: both weighted and binary versions of the selected graph family are preserved for later analysis under the same chosen representation.

0.8.1 Outputs of this Step

The implemented Step 8 writes its selection outputs under `outputs/representation_selection`. The current file set is:

- `08_representation_decision_matrix.csv`
- `08_representation_scenarios.csv`
- `08_unit_level_consistency_audit.csv`
- `08_primary_representation_selection.csv`
- `08_representation_selection_decision_log.csv`

The decision-matrix table contains one row per family in the formal decision set $\mathcal{R}_{\text{dec}}$. The scenario table contains the three transparency totals and their summary statistics. The consistency-audit table reports unit-class dominance shares for the pooled-reference induced subgraphs together with an overall consistency row for each family. The one-row selection table freezes the primary representation family used in later steps.

0.8.2 Implementation Note and Current Run

The current Step 8 implementation uses the available family set

$$\mathcal{R}_{\text{avail}} = \{\text{name_only}, \text{concat}\},$$

with `concat` supplied from the Step 6 source representation `name_description`. The family `fusion` is not available in the current run.

Under the pooled usability screen inherited from Step 7, neither available family is eligible:

- `name_only`: $\lambda_r = 0.0791$, $\iota_r = 0.4812$
- `concat`: $\lambda_r = 0.2681$, $\iota_r = 0.2534$

Since both families fail the $(\lambda_{\min}, \iota_{\max})$ screen, the current run is explicitly a fallback selection over all available families.

The formal pooled decision matrix in the current run yields:

- **concat**: structural score = 1.0000, parsimony score = 0.0000, composite score = 0.7500, final rank = 1
- **name_only**: structural score = 0.0000, parsimony score = 1.0000, composite score = 0.2500, final rank = 2

The scenario table is fully concordant in the current run:

- **concat**: richness = 0.85, simplicity = 0.60, balanced = 0.75, scenario wins = 3
- **name_only**: richness = 0.15, simplicity = 0.40, balanced = 0.25, scenario wins = 0

The unit-level consistency audit also strongly favors **concat**. Its overall audit score is 0.9985, compared with 0.0548 for **name_only**. In the current pooled-reference induced graphs, **concat** dominates all five audit metrics across sections and tracks, all five across programmes, and all but the largest-component-share criterion across programme-year units.

The resulting frozen primary representation record in the current run is therefore:

- `selected_representation_family = concat`
- `selected_source_representation = name_description`
- `fallback_decision = TRUE`
- `sensitivity_families = name_only`

This means that all subsequent mainline analyses should use the **concat** graph family exported from Step 7, while **name_only** is retained only as a sensitivity family.

0.9 Step 9: Descriptive profiling and overlap analysis on the selected graph family

Once the primary representation family has been selected in Step 8, the next stage is to construct a descriptive atlas of the resulting curriculum graphs before moving to higher-order topological interpretation. The purpose of this step is to summarize the observable structure of the ICT curricula across the available analytical units and to quantify the extent to which skills and retained relations are shared across years, programmes, sections, and tracks.

All descriptive analyses in this step are performed on the frozen primary graph family

$$\mathcal{H}^{(r^*)} = \left\{ H_{\text{all}}^{(r^*)}, H_t^{(r^*)}, H_s^{(r^*)}, H_p^{(r^*)}, H_{p,y}^{(r^*)} \right\},$$

where $H_{\text{all}}^{(r^*)}$ is the pooled sparse graph selected in Step 8, and the remaining graphs are pooled-reference induced subgraphs at the track, section, programme, and programme-year levels. In what follows, programme-year graphs remain the primary observational layer, while the higher levels are used for aggregation and synthesis.

Let

$$H_A = (V_A, E_A, w_A)$$

denote the graph associated with an analytical unit A , where V_A is the set of canonical skills present in A , E_A is the retained undirected edge set, and w_A is the corresponding weight function inherited from the selected pooled sparse family.

(i) Basic graph profile. The first descriptive block records the basic size and sparsity characteristics of each graph. For any analytical unit A , define

$$n(A) = |V_A|, \quad m(A) = |E_A|.$$

The binary density is

$$\text{dens}(A) = \frac{2m(A)}{n(A)(n(A) - 1)},$$

for $n(A) \geq 2$, and zero otherwise. The mean degree is

$$\bar{k}(A) = \frac{2m(A)}{n(A)}.$$

For a node $i \in V_A$, let

$$k_i(A)$$

denote its degree and

$$s_i(A) = \sum_{j: \{i,j\} \in E_A} w_{A,ij}$$

its weighted strength. The mean strength is

$$\bar{s}(A) = \frac{1}{n(A)} \sum_{i \in V_A} s_i(A).$$

The isolate share is

$$\text{iso}(A) = \frac{1}{n(A)} \sum_{i \in V_A} \mathbb{1}(k_i(A) = 0),$$

and the connected-node share is

$$\text{conn}(A) = 1 - \text{iso}(A).$$

In addition, let

$$C_1(A)$$

denote the largest connected component of H_A . Its relative size is

$$\text{LCC}(A) = \frac{|C_1(A)|}{n(A)}.$$

These quantities provide a first descriptive summary of how large, sparse, and connected each curriculum graph is.

(ii) Skill recurrence across years within programmes. Because the skills retain their curricular provenance, the second descriptive block summarizes how knowledge is distributed over time within each programme. Let $\mathcal{Y}_p$ denote the year set of programme p , and let

$$V_p = \bigcup_{y \in \mathcal{Y}_p} V_{p,y}$$

be the complete programme-level skill inventory. For each skill $i \in V_p$, define its within-programme recurrence count as

$$f_p(i) = \sum_{y \in \mathcal{Y}_p} \mathbb{1}(i \in V_{p,y}).$$

This count records in how many curricular years skill i appears within programme p . The share of recurrent skills in programme p is then

$$\text{rec}(p) = \frac{1}{|V_p|} \sum_{i \in V_p} \mathbb{1}(f_p(i) \geq 2).$$

This quantity distinguishes programmes that revisit previously introduced skills from those whose yearly content is more weakly overlapping.

(iii) Year-to-year overlap within programmes. To study progression inside programmes, overlap is first evaluated across years within the same programme. For two years $y_1, y_2 \in \mathcal{Y}_p$, the node-level overlap is measured by the Jaccard coefficient

$$J_V((p, y_1), (p, y_2)) = \frac{|V_{p,y_1} \cap V_{p,y_2}|}{|V_{p,y_1} \cup V_{p,y_2}|}.$$

To distinguish continuity from renewal, the carryover rate from y_1 to y_2 is defined as

$$\text{carry}((p, y_1) \rightarrow (p, y_2)) = \frac{|V_{p,y_1} \cap V_{p,y_2}|}{|V_{p,y_1}|},$$

while the introduction rate in y_2 relative to y_1 is

$$\text{intro}((p, y_1) \rightarrow (p, y_2)) = \frac{|V_{p,y_2} \setminus V_{p,y_1}|}{|V_{p,y_2}|}.$$

Together, these quantities provide a descriptive account of curricular persistence versus novelty over time.

(iv) Cross-unit node overlap. To compare curricula at the same or different levels of aggregation, node overlap is computed for any pair of analytical units A and B . The symmetric overlap is

$$J_V(A, B) = \frac{|V_A \cap V_B|}{|V_A \cup V_B|},$$

while the directional inclusion rate is

$$\text{incl}(A \rightarrow B) = \frac{|V_A \cap V_B|}{|V_A|}.$$

The Jaccard coefficient quantifies the overall proportion of shared skills, whereas the inclusion rate measures the extent to which the smaller or more specific unit is contained in the other. These metrics are especially useful when comparing programme-years at the same curricular stage, programmes within the same track, sections across tracks, and each lower-level unit against the pooled universe.

(v) Edge overlap conditional on shared skills. Node overlap alone does not show whether two curricula organize their shared skills in similar ways. For two analytical units A and B , define the shared skill set

$$S_{A,B} = V_A \cap V_B.$$

Let

$$E_A^S = \{\{i, j\} \in E_A : i, j \in S_{A,B}\}, \quad E_B^S = \{\{i, j\} \in E_B : i, j \in S_{A,B}\}.$$

The conditional edge overlap is then

$$J_E(A, B) = \frac{|E_A^S \cap E_B^S|}{|E_A^S \cup E_B^S|},$$

with the convention that $J_E(A, B) = 0$ whenever the denominator is zero. This measure answers a stronger question than node overlap: among the skills shared by two curricula, to what extent are the retained relations also shared?

(vi) Weighted agreement on shared relations. Because the selected family is available in weighted form, descriptive comparison also includes the agreement in edge strength on the common retained support. Let

$$E_{A,B}^\cap = E_A^S \cap E_B^S$$

be the set of shared retained edges between A and B after restricting to shared skills. Let $\mathbf{w}_A$ and $\mathbf{w}_B$ denote the vectors of edge weights on $E_{A,B}^\cap$ in the two graphs. Whenever at least two shared edges are available, weighted agreement is summarized by the Spearman correlation

$$\rho_w(A, B) = \rho_S(\mathbf{w}_A, \mathbf{w}_B).$$

This coefficient captures whether the same retained relations tend to be similarly strong across curricula, even if the overall edge sets are not identical.

(vii) Overlap summaries by unit class. The overlap statistics above are aggregated into comparison matrices for the main analytical classes. Let

$$\mathcal{U} = \{\text{tracks}, \text{sections}, \text{programmes}, \text{programme_years}\}.$$

For each class $u \in \mathcal{U}$, let $\mathcal{A}_u$ be the set of units of that class. Then Step 9 produces, for each u ,

$$[J_V(A, B)]_{A, B \in \mathcal{A}_u}, \quad [J_E(A, B)]_{A, B \in \mathcal{A}_u}, \quad [\rho_w(A, B)]_{A, B \in \mathcal{A}_u},$$

together with directional inclusion matrices

$$[\text{incl}(A \rightarrow B)]_{A,B \in \mathcal{A}_u}.$$

At the programme-year level, these matrices are also stratified by curricular year when appropriate, so that same-stage comparisons are not confounded with year aggregation.

(viii) Outputs of the descriptive atlas. The outputs of Step 9 are organized into four descriptive blocks. First, a graph profile table records, for every analytical unit, the node count, edge count, density, mean degree, mean strength, isolate share, connected-node share, and largest-component share. Second, a recurrence table summarizes within-programme skill persistence across years. Third, overlap and inclusion matrices quantify shared skill coverage across tracks, sections, programmes, and programme-years. Fourth, conditional edge-overlap and weighted-agreement matrices describe the similarity of retained relational structure across units. The aim of Step 9 is therefore to establish a transparent descriptive baseline on the selected graph family before introducing the more formal network measures of the next stage. By separating graph size, recurrence, content overlap, and relational agreement, this step provides the empirical foundation for the subsequent analysis of clustering, modularity, centrality, assortativity, bridge structure, and curriculum-distance metrics.

0.9.1 Outputs of this Step

The implemented Step 9 writes its descriptive-atlas outputs under `outputs/descriptive_atlas`. The current file set is:

- `09_graph_profile_table.csv`
- `09_programme_skill_recurrence_summary.csv`
- `09_programme_skill_recurrence_detail.csv`
- `09_programme_year_progression_overlap.csv`
- `09_overlap_matrix_tracks.csv`
- `09_overlap_matrix_sections.csv`
- `09_overlap_matrix_programmes.csv`
- `09_overlap_matrix_programme_years.csv`
- `09_overlap_to_pooled.csv`
- `09_descriptive_atlas_summary.csv`

The graph-profile table contains one row per analytical unit in the selected family. The recurrence files summarize within-programme skill persistence both at the programme level and at the individual-skill level. The overlap files are stored in long-format matrix form, with one row per unit pair and separate columns for node overlap, directional inclusion, conditional edge overlap, and weighted agreement.

0.9.2 Implementation Note and Current Run

The current Step 9 run uses the primary family frozen in Step 8:

- `selected_representation_family = concat`
- `selected_source_representation = name_description`
- `reference_threshold = 0.519089`

The implemented descriptive atlas currently contains:

- 63 graph-profile rows;
- 14 programme-level recurrence-summary rows;
- 862 programme-skill recurrence-detail rows;
- 29 within-programme year-to-year progression rows;
- 1 track overlap row;
- 78 section overlap rows;
- 91 programme overlap rows;
- 528 programme-year overlap rows;
- 62 lower-unit versus pooled comparison rows.

At the pooled all-tracks level, the current selected graph has:

- $n = 746$ nodes;
- $m = 739$ retained edges;
- $\text{density} = 0.002659$;
- $\text{mean degree} = 1.9812$;
- $\text{mean strength} = 1.1401$;
- $\text{isolate share} = 0.2534$;
- $\text{connected-node share} = 0.7466$;
- $\text{largest-component share} = 0.2681$.

The recurrence summary shows substantial heterogeneity across programmes in the current run. The highest recurrent-skill shares are:

- MAIOU: 0.4167
- SINFO: 0.2931
- TECNO: 0.1748

The progression table currently includes all unordered year pairs within each programme, not only adjacent years. The programme-year overlap table also includes same-year flags, same-programme flags, and year gaps so that same-stage comparisons across programmes can be separated from within-programme longitudinal comparisons.

One implementation detail should be noted explicitly. The overlap “matrices” are exported as long-format pair tables rather than wide square CSV matrices, following the same storage logic used earlier for similarity matrices. This keeps the outputs easier to join, filter, and stratify by year or programme in downstream analysis.